

ASSIGNMENT 5.5(AI ASST CODING)

#TASK-1

Task: Use AI to generate two solutions for checking prime numbers:

- Naive approach(basic)
- Optimized approach

PROMPT-

Generate Python code for two prime-checking methods and explain how the optimized version improves performance

Code:

```
def is_prime_basic(num):
>     if num < 2: ...
        for i in range(2, num):
            if num % i == 0:
                return False
        return True
def is_prime_optimized(num):
    if num < 2:
        return False
    if num == 2:
        return True
    if num % 2 == 0:
        return False
    for i in range(3, int(num ** 0.5) + 1, 2):
        if num % i == 0:
            return False
    return True

# Example usage
number = int(input("Enter a number: "))
if is_prime_optimized(number):
    print("It is a prime number")
else:
    print("It is not a prime number")
```

Output:

```
Enter a number: 16  
It is not a prime number  
PS C:\200126> 
```

Observation:

Both naive and optimized methods are used to check whether a number is prime.

The naive method checks divisibility with all numbers from 2 to $n-1$, which is simple but slow.

The optimized method checks only up to the square root of the number, reducing unnecessary checks.

Thus, the optimized method is more efficient, especially for large numbers.

#TASK-2

Objective: Use AI to generate a recursive function to calculate Fibonacci numbers.

1. Ask AI to add clear comments explaining recursion.
2. Ask AI to explain base cases and recursive calls.

PROMPT-

generate a recursive function to calculate Fibonacci numbers and also add clear comments to explain recursion and also explain base cases and recursive calls

CODE-

```
def fibonacci_recursive(n):  
    """  
    Recursive function to calculate the nth Fibonacci number.  
  
    Base cases: Handle the simplest scenarios where recursion stops.  
    - If n <= 0, return 0 (0th Fibonacci number)  
    - If n == 1, return 1 (1st Fibonacci number)  
  
    Recursive case: Break down the problem into smaller subproblems.  
    - fib(n) = fib(n-1) + fib(n-2)  
    - Each call reduces the problem size until base cases are reached  
    """  
  
    # Base case 1: Return 0 for n <= 0  
    if n <= 0:  
        return 0  
  
    # Base case 2: Return 1 for n == 1  
    if n == 1:  
        return 1  
  
    # Recursive case: Sum of two previous Fibonacci numbers  
    # This breaks the problem into smaller subproblems  
    return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)  
  
    # Example usage  
num = int(input("Enter a position in Fibonacci sequence: "))  
result = fibonacci_recursive(num)  
print(f"The {num}th Fibonacci number is: {result}")
```

OUTPUT-

```
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment5_5.py  
Enter a position in Fibonacci sequence: 32  
The 32th Fibonacci number is: 2178309  
PS C:\200126> |
```

OBSERVATION-

- 1)The recursive Fibonacci program works by breaking a big problem into smaller, similar problems.
- 2)The **base calls** handle the simplest cases (like Fibonacci of 0 or 1) and stop the recursion.
- 3)For other values, the function makes **recursive calls** to compute the previous two Fibonacci numbers.
- 4)Each call waits for its smaller results and then combines them, showing how recursion builds the final answer step by step.

#TASK-3

TASK:

Use AI to generate a Python program that reads a file and processes data.

PROMPT:

Generate code with proper error handling and clear explanations for each exception.

CODE:

```
def read_and_process_file(filename):
    """
    Reads a file and processes data with comprehensive error handling.

    Handles multiple exception types:
    - FileNotFoundError: File doesn't exist
    - IOError: File read errors
    - ValueError: Invalid data format
    - Exception: Catch-all for unexpected errors
    """
    try:
        # Attempt to open and read the file
        with open(filename, 'r') as file:
            lines = file.readlines()

        # Process each line
        numbers = []
        for line in lines:
            try:
                # Convert line to integer and process
                num = int(line.strip())
                numbers.append(num)
            except ValueError:
                # Handle non-integer data
                print(f"Warning: Skipping invalid data '{line.strip()}'")
                continue

        # Calculate and display results
        if numbers:
            print(f"Sum: {sum(numbers)}")
            print(f"Average: {sum(numbers) / len(numbers):.2f}")
        else:
            print("No valid numbers found in file")

    except FileNotFoundError:
        # Handle missing file
        print(f"Error: File '{filename}' not found")
    except IOError as e:
        # Handle file read errors
        print(f"Error: Cannot read file - {e}")
    except Exception as e:
        # Catch unexpected errors
        print(f"Unexpected error: {e}")

# Example usage
filename = input("Enter filename: ")
read_and_process_file(filename)
```

OUTPUT:

PS C:\200126> &

C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe
c:/200126/assignment5_5.py

Enter filename: _____

Here the code is in user input format so the file name varies

OBSERVATION:

The AI-generated Python program successfully reads data from a file and processes it in a structured manner.

It simplifies file handling by automating repetitive tasks such as reading, parsing, and data manipulation.

This approach reduces coding effort and minimizes errors while improving efficiency. Overall, using AI helps in quickly generating reliable code for file processing tasks.

#TASK-4

TASK: Use an AI tool to generate a Python-based login system.

PROMPT:

Generate a python based login system to check whether the AI uses secure password handling practices.

CODE:

```

def hash_password(password):
    """Hash password using SHA256"""
    return sha256(password.encode()).hexdigest()

def validate_password_strength(password):
    """Check if password meets security requirements"""
    if len(password) < 8:
        return False, "Password must be at least 8 characters"
    if not re.search(r'[A-Z]', password):
        return False, "Must contain uppercase letter"
    if not re.search(r'[a-z]', password):
        return False, "Must contain lowercase letter"
    if not re.search(r'[0-9]', password):
        return False, "Must contain digit"
    if not re.search(r'[!@#%&*]', password):
        return False, "Must contain special character"
    return True, "Password is strong"

def login_system():
    """Secure login system with password validation"""
    users = {}

    while True:
        choice = input("\n1. Register\n2. Login\n3. Exit\nChoice: ").strip()

        if choice == "1":
            username = input("Username: ").strip()
            if username in users:
                print("User already exists")
                continue

            password = input("Password: ")
            is_valid, message = validate_password_strength(password)
            print(message)

            if is_valid:
                users[username] = hash_password(password)
                print("Registration successful")

        elif choice == "2":
            username = input("Username: ").strip()
            password = input("Password: ")

            if username in users and users[username] == hash_password(password):
                print(f"Welcome {username}!")
            else:
                print("Invalid credentials")

        elif choice == "3":
            break

    login_system()

```

OUTPUT:

```

PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment5_5.py

1. Register
2. Login
3. Exit
Choice:

```

OBSERVATION:

The Python-based login system checks whether secure methods are used to handle passwords. Instead of saving passwords directly, the system uses basic security techniques to protect user data. This helps prevent unauthorized access and shows the importance of following safe practices while developing AI-based login systems

#TASK-5

TASK- Use an AI tool to generate a Python script that logs user activity (username, IP address, timestamp).

PROMPT:

generate a Python script that logs user activity (username, IP address, timestamp) to examine whether sensitive data is logged unnecessarily or insecurely.

CODE:

```
#generate a python code that logs user activity to examine whether sensitive data is logged unnecessarily or insecurely
def login(username, password):
    print("Login attempt detected")

    # Do NOT print sensitive data
    # print(f"Username: {username}, Password: {password}")

    if authenticate_user(username, password):
        print(f"Login successful for user: {username}")
        return True
    else:
        print(f"Login failed for user: {username}")
        return False

def authenticate_user(username, password):
    # Dummy authentication logic
    users = {"admin": "hashed_password"} # password assumed to be hashed
    return username in users

def logout(username):
    print(f"User logged out: {username}")

# Simulated user activity
login("admin", "mypassword123")
logout("admin")
```

OUTPUT:

```
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment5_5.py
Login attempt detected
Login successful for user: admin
User logged out: admin
PS C:\200126> █
```

OBSERVATION:

The program prints user activity such as login and logout actions directly in the terminal without displaying sensitive information like passwords. This shows that user data is handled carefully and unnecessary or insecure details are not exposed during logging.

